

Figure 21.4 Virtual-to-physical address translation on IA-32.

only 8 MB. Additionally, the MM allocates pages of PDEs and PTEs as needed and moves page-table pages to secondary storage when not in use, so that the actual physical memory overhead of the paging structures for each process is usually approximately 2 KB. The page-table pages are faulted back into memory when referenced.

We next consider how virtual addresses are translated into physical addresses on IA-32-compatible processors. A 2-bit value can represent the values 0, 1, 2, 3. A 9-bit value can represent values from 0 to 511; a 12-bit value, values from 0 to 4,095. Thus, a 12-bit value can select any byte within a 4-KB page of memory. A 9-bit value can represent any of the 512 PDEs or PTEs in a page directory or PTE-table page. As shown in Figure 21.4, translating a virtual address pointer to a byte address in physical memory involves breaking the 32-bit pointer into four values, starting from the most significant bits:

- Two bits are used to index into the four PDEs at the top level of the page table. The selected PDE will contain the physical page number for each of the four page-directory pages that map 1 GB of the address space.
- Nine bits are used to select another PDE, this time from a second-level page directory. This PDE will contain the physical page numbers of up to 512 PTE-table pages.
- Nine bits are used to select one of 512 PTEs from the selected PTE-table page. The selected PTE will contain the physical page number for the byte we are accessing.
- Twelve bits are used as the byte offset into the page. The physical address of the byte we are accessing is constructed by appending the lowest 12 bits of the virtual address to the end of the physical page number we found in the selected PTE.

Note that the number of bits in a physical address may be different from the number of bits in a virtual address. For example, when PAE is enabled (the only mode supported by Windows 8 and later versions), the IA-32 MMU is extended to the larger 64-bit PTE size, while the hardware supports 36-bit physical addresses, granting access to up to 64 GB of RAM, even though a single process can only map an address space up to 4 GB in size. Today, on the AMD64 architecture, server versions of Windows support very, very large physical addresses—more than we can possibly use or even buy (24 TB as of the latest release). (Of course, at one time time 4 GB seemed optimistically large for physical memory.)

To improve performance, the MM maps the page-directory and PTE-table pages into the same contiguous region of virtual addresses in every process. This self-map allows the MM to use the same pointer to access the current PDE or PTE corresponding to a particular virtual address no matter what process is running. The self-map for the IA-32 takes a contiguous 8-MB region of kernel virtual address space; the AMD64 self-map occupies 512 GB. Although the self-map occupies significant address space, it does not require any additional virtual memory pages. It also allows the page table's pages to be automatically paged in and out of physical memory.

In the creation of a self-map, one of the PDEs in the top-level page directory refers to the page-directory page itself, forming a "loop" in the page-table translations. The virtual pages are accessed if the loop is not taken, the PTE-table pages are accessed if the loop is taken once, the lowest-level page-directory pages are accessed if the loop is taken twice, and so forth.

The additional levels of page directories used for 64-bit virtual memory are translated in the same way except that the virtual address pointer is broken up into even more values. For the AMD64, Windows uses four full levels, each of which maps 512 pages, or 9 + 9 + 9 + 9 + 12 = 48 bits of virtual address.

To avoid the overhead of translating every virtual address by looking up the PDE and PTE, processors use **translation look-aside buffer** (**TLB**) hardware, which contains an associative memory cache for mapping virtual pages to PTEs. The TLB is part of the **memory-management unit** (**MMU**) within each processor. The MMU needs to "walk" (navigate the data structures of) the page table in memory only when a needed translation is missing from the TLB.

The PDEs and PTEs contain more than just physical page numbers. They also have bits reserved for operating-system use and bits that control how the hardware uses memory, such as whether hardware caching should be used for each page. In addition, the entries specify what kinds of access are allowed for both user and kernel modes.

A PDE can also be marked to say that it should function as a PTE rather than a PDE. On a IA-32, the first 11 bits of the virtual address pointer select a PDE in the first two levels of translation. If the selected PDE is marked to act as a PTE, then the remaining 21 bits of the pointer are used as the offset of the byte. This results in a 2-MB size for the page. Mixing and matching 4-KB and 2-MB page sizes within the page table is easy for the operating system and can significantly improve the performance of some programs. The improvement results from reducing how often the MMU needs to reload entries in the TLB, since one PDE mapping 2 MB replaces 512 PTEs, each mapping 4 KB. Newer AMD64 hardware even supports 1-GB pages, which operate in a similar fashion.

Managing physical memory so that 2-MB pages are available when needed is difficult, as they may continually be broken up into 4-KB pages, causing external fragmentation of memory. Also, the large pages can result in very significant internal fragmentation. Because of these problems, it is typically only Windows itself, along with large server applications, that use large pages to improve the performance of the TLB. They are better suited to do so because operating-system and server applications start running when the system boots, before memory has become fragmented.

Windows manages physical memory by associating each physical page with one of seven states: free, zeroed, modified, standby, bad, transition, or valid.

- A *free* page is an available page that has stale or uninitialized content.
- A *zeroed* page is a free page that has been zeroed out and is ready for immediate use to satisfy zero-on-demand faults.
- A *modified* page has been written by a process and must be sent to secondary storage before it is usable by another process.
- A *standby* page is a copy of information already stored on secondary storage. Standby pages may be pages that were not modified, modified pages that have already been written to secondary storage, or pages that were prefetched because they were expected to be used soon.
- A *bad* page is unusable because a hardware error has been detected.
- A *transition* page is on its way from secondary storage to a page frame allocated in physical memory.
- A *valid* page either is part of the working set of one or more processes and is contained within these processes' page tables, or is being used by the system directly (such as to store the nonpaged pool).

While valid pages are contained in processes' page tables, pages in other states are kept in separate lists according to state type. Additionally, to improve performance and protect against aggressive recycling of the standby pages, Windows Vista and later versions implement eight prioritized standby lists. The lists are constructed by linking the corresponding entries in the **page frame number** (**PFN**) database, which includes an entry for each physical memory page. The PFN entries also include information such as reference counts, locks, and NUMA information. Note that the PFN database represents pages of physical memory, whereas the PTEs represent pages of virtual memory.

When the valid bit in a PTE is zero, hardware ignores all the other bits, and the MM can define them for its own use. Invalid pages can have a number of states represented by bits in the PTE. Page-file pages that have never been faulted in are marked zero-on-demand. Pages mapped through section objects encode a pointer to the appropriate section object. PTEs for pages that have been written to the page file contain enough information to locate the page on secondary storage, and so forth. The structure of the page-file PTE is shown in Figure 21.5. The T, P, and V bits are all zero for this type of PTE. The PTE includes 5 bits for page protection, 32 bits for page-file offset, and 4 bits to select the paging file. There are also 20 bits reserved for additional bookkeeping.

Windows uses a per-working-set, least recently used (LRU) replacement policy to take pages from processes as appropriate. When a process is started, it is assigned a default minimum working-set size, at which point the MM starts to track the age of the pages in each working set. The working set of each process is allowed to grow until the amount of remaining physical memory starts to run low. Eventually, when the available memory runs critically low, the MM trims the working set to remove older pages.

The age of a page depends not on how long it has been in memory but on when it was last referenced. The MM makes this determination by periodically passing through the working set of each process and incrementing the age for pages that have not been marked in the PTE as referenced since the last pass. When it becomes necessary to trim the working sets, the MM uses heuristics to

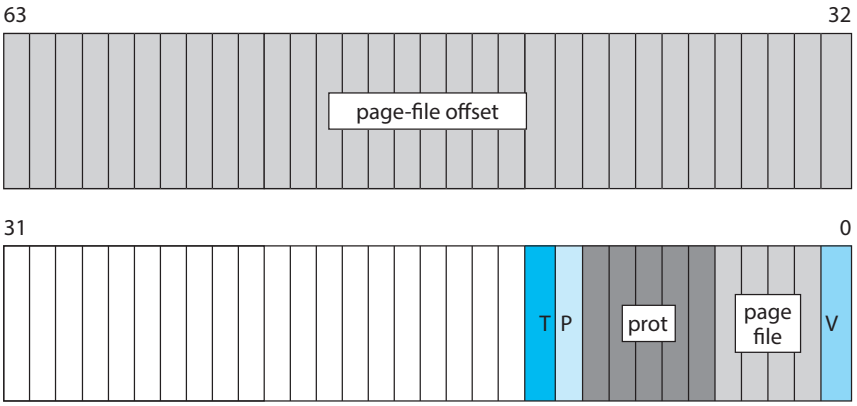


Figure 21.5 Page-file page-table entry. The valid bit is zero.

decide how much to trim from each process and then removes the oldest pages first.

A process can have its working set trimmed even when plenty of memory is available, if it was given a ***hard limit*** on how much physical memory it could use. In Windows 7 and later versions, the MM also trims processes that are growing rapidly, even if memory is plentiful. This policy change significantly improved the responsiveness of the system for other processes.

Windows tracks working sets not only for user-mode processes but also for various kernel-mode regions, which include the file cache and the pageable kernel heap. Pageable kernel and driver code and data have their own working sets, as does each TS session. The distinct working sets allow the MM to use different policies to trim the different categories of kernel memory.

The MM does not fault in only the page immediately needed. Research shows that the memory referencing of a thread tends to have a **locality** property. That is, when a page is used, it is likely that adjacent pages will be referenced in the near future. (Think of iterating over an array or fetching sequential instructions that form the executable code for a thread.) Because of locality, when the MM faults in a page, it also faults in a few adjacent pages. This prefetching tends to reduce the total number of page faults and allows reads to be clustered to improve I/O performance.

In addition to managing committed memory, the MM manages each process's reserved memory, or virtual address space. Each process has an associated tree that describes the ranges of virtual addresses in use and what the uses are. This allows the MM to fault in page-table pages as needed. If the PTE for a faulting address is uninitialized, the MM searches for the address in the process's tree of **virtual address descriptors** (**VADs**) and uses this information to fill in the PTE and retrieve the page. In some cases, a PTE-table page may not exist; such a page must be transparently allocated and initialized by the MM. In other cases, the page may be shared as part of a section object, and the VAD will contain a pointer to that section object. The section object contains information on how to find the shared virtual page so that the PTE can be initialized to point to it directly.

Starting with Vista, the Windows MM includes a component called SuperFetch. This component combines a user-mode service with specialized kernel-

mode code, including a file-system filter, to monitor all paging operations on the system. Each second, the service queries a trace of all such operations and uses a variety of agents to monitor application launches, fast user switches, standby/sleep/hibernate operations, and more as a means of understanding the system's usage patterns. With this information, it builds a statistical model, using Markov chains, of which applications the user is likely to launch when, in combination with what other applications, and what portions of these applications will be used. For example, SuperFetch can train itself to understand that the user launches Microsoft Outlook in the mornings mostly to read e-mail but composes e-mails later, after lunch. It can also understand that once Outlook is in the background, Visual Studio is likely to be launched next, and that the text editor is going to be in high demand, with the compiler demanded a little less frequently, the linker even less frequently, and the documentation code hardly ever. With this data, SuperFetch will prepopulate the standby list, making low-priority I/O reads from secondary storage at idle times to load what it thinks the user is likely to do next (or another user, if it knows a fast user switch is likely). Additionally, by using the eight prioritized standby lists that Windows offers, each such prefetched paged can be cached at a level that matches the statistical likelihood that it will be needed. Thus, unlikely-to-be-demanded pages can cheaply and quickly be evicted by an unexpected need for physical memory, while likely-to-be-demanded-soon pages can be kept in place for longer. Indeed, SuperFetch may even force the system to trim working sets of other processes before touching such cached pages.

SuperFetch's monitoring does create considerable system overhead. On mechanical (rotational) drives, which have seek times in the milliseconds, this cost is balanced by the benefit of avoiding latencies and multisecond delays in application launch times. On server systems, however, such monitoring is not beneficial, given the random multiuser workloads and the fact that throughput is more important than latency. Further, the combined latency improvements and bandwidth on systems with fast, efficient nonvolatile memory, such as SSDs, make the monitoring less beneficial for those systems as well. In such situations, SuperFetch disables itself, freeing up a few spare CPU cycles.

Windows 10 brings another large improvement to the MM by introducing a component called the compression store manager. This component creates a compressed store of pages in the working set of the **memory compression process**, which is a type of system process. When shareable pages go on the standby list and available memory is low (or certain other internal algorithm decisions are made), pages on the list will be compressed instead of evicted. This can also happen to modified pages targeted for eviction to secondary storage—both by reducing memory pressure, perhaps avoiding the write in the first place, and by causing the written pages to be compressed, thus consuming less page file space and taking less I/O to page out. On today's fast multiprocessor systems, often with built-in hardware compression algorithms, the small CPU penalty is highly preferable to the potential secondary storage I/O cost.

21.3.5.3 Process Manager

The Windows process manager provides services for creating, deleting, interrogating, and managing processes, threads, and jobs. It has no knowledge

about parent–child relationships or process hierarchies, although it can group processes in jobs, and the latter can have hierarchies that must then be maintained. The process manager is also not involved in the scheduling of threads, other than setting the priorities and affinities of the threads in their owner processes. Additionally, through jobs, the process manager can effect various changes in scheduling attributes (such as throttling ratios and quantum values) on threads. Thread scheduling proper, however, takes place in the kernel dispatcher.

Each process contains one or more threads. Processes themselves can be collected into larger units called **job objects**. The original use of job objects was to place limits on CPU usage, working-set size, and processor affinities that control multiple processes at once. Job objects were thus used to manage large data-center machines. In Windows XP and later versions, job objects were extended to provide security-related features, and a number of third-party applications such as Google Chrome began using jobs for this purpose. In Windows 8, a massive architectural change allowed jobs to influence scheduling through generic CPU throttling as well as per-user-session-aware fairness throttling/balancing. In Windows 10, throttling support was extended to secondary storage I/O and network I/O as well. Additionally, Windows 8 allowed job objects to nest, creating hierarchies of limits, ratios, and quotas that the system must accurately compute. Additional security and power management features were given to job objects as well.

As a result, all Windows Store applications and all UWP application processes run in jobs. The DAM, introduced earlier, implements Connected Standby support using jobs. Finally, Windows 10's support for Docker Containers, a key part of its cloud offerings, uses job objects, which it calls **silos**. Thus, jobs have gone from being an esoteric data-center resource management feature to a core mechanism of the process manager for multiple features.

Due to Windows's layered architecture and the presence of environment subsystems, process creation is quite complex. An example of process creation in the Win32 environment under Windows 10 is as follows. Note that the launching of UWP "Modern" Windows Store applications (which are called **packaged applications**, or "AppX") is significantly more complex and involves factors outside the scope of this discussion.

1. A Win32 application calls `CreateProcess()`.
2. A number of parameter conversions and behavioral conversions are done from the Win32 world to the NT world.
3. `CreateProcess()` then calls the `NtCreateUserProcess()` API in the process manager of the NT executive to actually create the process and its initial thread.
4. The process manager calls the object manager to create a process object and returns the object handle to Win32. It then calls the memory manager to initialize the address space of the new process, its handle table, and other key data structures, such as the process environment block (PEBL) (which contains internal process management data).

5. The process manager calls the object manager again to create a thread object and returns the handle to Win32. It then calls the memory manager to create the thread environment block (TEB) and the dispatcher to initialize the scheduling attributes of the thread, setting its state to `initializing`.
6. The process manager creates the initial thread startup context (which will eventually point to the `main()` routine of the application), asks the scheduler to mark the thread as `ready`, and then immediately suspends it, putting it into a `waiting` state.
7. A message is sent to the Win32 subsystem to notify it that the process is being created. The subsystem performs additional Win32-specific work to initialize the process, such as computing its shutdown level and drawing the animated hourglass or "donut" mouse cursor.
8. Back in `CreateProcess()`, inside the parent process, the `ResumeThread()` API is called to wake up the process's initial thread. Control returns to the parent.
9. Now, inside the initial thread of the new process, the user-mode link loader takes control (inside `ntdll.dll`, which is automatically mapped into all processes). It loads all the library dependencies (DLLs) of the application, creates its initial heap, sets up exception handling and application compatibility options, and eventually calls the `main()` function of the application.

The Windows APIs for manipulating virtual memory and threads and for duplicating handles take a process handle, so their subsystem and other services, when notified of process creation, can perform operations on behalf of the new process without having to execute directly in the new process's context. Windows also supports a UNIX `fork()` style of process creation. A number of features—including **process reflectio** , which is used by the Windows error reporting (WER) infrastructure during process crashes, as well as the Windows subsystem for Linux's implementation of the Linux `fork()` API—depend on this capability.

The debugger support in the process manager includes the APIs to suspend and resume threads and to create threads that begin in suspended mode. There are also process-manager APIs that get and set a thread's register context and access another process's virtual memory. Threads can be created in the current process; they can also be injected into another process. The debugger makes use of thread injection to execute code within a process being debugged. Unfortunately, the ability to allocate, manipulate, and inject both memory and threads across processes is often misused by malicious programs.

While running in the executive, a thread can temporarily attach to a different process. **Thread attach** is used by kernel worker threads that need to execute in the context of the process originating a work request. For example, the MM might use thread attach when it needs access to a process's working set or page tables, and the I/O manager might use it in updating the status variable in a process for asynchronous I/O operations.

21.3.5.4 Facilities for Client–Server Computing

Like many other modern operating systems, Windows uses a client–server model throughout, primarily as a layering mechanism, which allows putting common functionality into a "service" (the equivalent of a daemon in UNIX terms), as well as splitting out content-parsing code (such as a PDF reader or Web browser) from system-action-capable code (such as the Web browser's capability to save a file on secondary storage or the PDF reader's ability to print out a document). For example, on a recent Windows 10 operating system, opening the New York Times website with the Microsoft Edge browser will likely result in 12 to 16 different processes in a complex organization of "broker," "renderer/parser," "JITTer," services, and clients.

The most basic such "server" on a Windows computer is the Win32 environment subsystem, which is the server that implements the operating-system personality of the Win32 API inherited from the Windows 95/98 days. Many other services, such as user authentication, network facilities, printer spooling, Web services, network file systems, and plug-and-play, are also implemented using this model. To reduce the memory footprint, multiple services are often collected into a few processes running the `svchost.exe` program. Each service is loaded as a dynamic-link library (DLL), which implements the service by relying on user-mode thread-pool facilities to share threads and wait for messages (see Section 21.3.5.3). Unfortunately, this pooling originally resulted in poor user experience in troubleshooting and debugging runaway CPU usage and memory leaks, and it weakened the overall security of each service. Therefore, in recent versions of Windows 10, if the system has over 2 GB of RAM, each DLL service runs in its own individual `svchost.exe` process.

In Windows, the recommended paradigm for implementing client–server computing is to use RPCs to communicate requests, because of their inherent security, serialization services, and extensibility features. The Win32 API supports the Microsoft standard of the DCE-RPC protocol, called MS-RPC, as described in Section 21.6.2.7.

RPC uses multiple transports (for example, named pipes and TCP/IP) that can be used to implement RPCs between systems. When an RPC occurs only between a client and a server on the local system, ALPC can be used as the transport. Furthermore, because RPC is heavyweight and has multiple system-level dependencies (including the WINXXIII environment subsystem itself), many native Windows services, as well as the kernel, directly use ALPC, which is not available (nor suitable) for third-party programmers.

ALPC is a message-passing mechanism similar to UNIX domain sockets and Mach IPC. The server process publishes a globally visible connection-port object. When a client wants services from the server, it opens a handle to the server's connection-port object and sends a connection request to the port. If the server accepts the connection, then ALPC creates a pair of communication-port objects, providing the client's connect API with its handle to the pair, and then providing the server's accept API with the other handle to the pair.

At this point, messages can be sent across communication ports as either `datagrams`, which behave like UDP and require no reply, or `requests`, which must receive a reply. The client and server can then use either synchronous messaging, in which one side is always blocking (waiting for a request or expecting a reply), or asynchronous messaging, in which the thread-pool

mechanism can be used to perform work whenever a request or reply is received, without the need for a thread to block for a message. For servers located in kernel mode, communication ports also support a callback mechanism, which allows an immediate switch to the kernel side (KT) of the user-mode thread (UT), immediately executing the server's handler routine.

When an ALPC message is sent, one of two message-passing techniques can be chosen.

1. The first technique is suitable for small to medium-sized messages (below 64 KB). In this case, the port's kernel message queue is used as intermediate storage, and the messages are copied from one process, to the kernel, to the other process. The disadvantage of this technique is the double buffering, as well as the fact that messages remain in kernel memory until the intended receiver consumes them. If the receiver is highly contended or currently unavailable, this may result in megabytes of kernel-mode memory being locked up.
2. The second technique is for larger messages. In this case, a shared-memory section object is created for the port. Messages sent through the port's message queue contain a "message attribute," called a **data view attribute**, that refers to the section object. The receiving side "exposes" this attribute, resulting in a virtual address mapping of the section object and a sharing of physical memory. This avoids the need to copy large messages or to buffer them in kernel-mode memory. The sender places data into the shared section, and the receiver sees them directly, as soon as it consumes a message.

Many other possible ways of implementing client–server communication exist, such as by using mailslots, pipes, sockets, section objects paired with events, window messages, and more. Each one has its uses, benefits, and disadvantages. RPC and ALPC remain the most fully featured, safe, secure, and feature-rich mechanisms for such communication, however, and they are the mechanisms used by the vast majority of Windows processes and services.

21.3.5.5 I/O Manager

The **I/O manager** is responsible for all device drivers on the system, as well as for implementing and defining the communication model that allows drivers to communicate with each other, with the kernel, and with user-mode clients and consumers. Additionally, as in UNIX-based operating systems, I/O is always targeted to a **fil object**, even if the device is not a file system. The I/O manager in Windows allows device drivers to be "filtered" by other drivers, creating a **device stack** through which I/O flows and which can be used to modify, extend, or enhance the original request. Therefore, the I/O manager always keeps track of which device drivers and filter drivers are loaded.

Due to the importance of file-system drivers, the I/O manager has special support for them and implements interfaces for loading and managing file systems. It works with the MM to provide memory-mapped file I/O and controls the Windows cache manager, which handles caching for the entire I/O system. The I/O manager is fundamentally asynchronous, providing synchronous I/O by explicitly waiting for an I/O operation to complete. The I/O manager

provides several models of asynchronous I/O completion, including setting of events, updating of a status variable in the calling process, delivery of APCs to initiating threads, and use of I/O completion ports, which allow a single thread to process I/O completions from many other threads. It also manages buffers for I/O requests.

Device drivers are arranged in a list for each device (called a driver or I/O stack). A driver is represented in the system as a **driver object**. Because a single driver can operate on multiple devices, the drivers are represented in the I/O stack by a **device object**, which contains a link to the driver object. Additionally, nonhardware drivers can use device objects as a way to expose different interfaces. As an example, there are TCP6, UDP6, UDP, TCP, RawIp, and RawIp6 device objects owned by the TCP/IP driver object, even though these don't represent physical devices. Similarly, each volume on secondary storage is its own device object, owned by the volume manager driver object.

Once a handle is opened to a device object, the I/O manager always creates a file object and returns a file handle instead of a device handle. It then converts the requests it receives (such as create, read, and write) into a standard form called an **I/O request packet** (**IRP**). It forwards the IRP to the first driver in the targeted I/O stack for processing. After a driver processes the IRP, it calls the I/O manager either to forward the IRP to the next driver in the stack or, if all processing is finished, to complete the operation represented by the IRP.

The I/O request may be completed in a context different from the one in which it was made. For example, if a driver is performing its part of an I/O operation and is forced to block for an extended time, it may queue the IRP to a worker thread to continue processing in the system context. In the original thread, the driver returns a status indicating that the I/O request is pending so that the thread can continue executing in parallel with the I/O operation. An IRP may also be processed in interrupt-service routines and completed in an arbitrary process context. Because some final processing may need to take place in the context that initiated the I/O, the I/O manager uses an APC to do final I/O-completion processing in the process context of the originating thread.

The I/O stack model is very flexible. As a driver stack is built, various drivers have the opportunity to insert themselves into the stack as **filte drivers**. Filter drivers can examine and potentially modify each I/O operation. Volume snapshotting (**shadow copies**) and disk encryption (**BitLocker**) are two built-in examples of functionality implemented using filter drivers that execute above the volume manager driver in the stack. File-system filter drivers execute above the file system and have been used to implement functionalities such as hierarchical storage management, single instancing of files for remote boot, and dynamic format conversion. Third parties also use file-system filter drivers to implement anti-malware tools. Due to the large number of file-system filters, Windows Server 2003 and later versions now include a **filte manager** component, which acts as the sole file-system filter and which loads **minifilter** ordered by specific **altitudes** (relative priorities). This model allows filters to transparently cache data and repeated queries without having to know about each other's requests. It also provides stricter load ordering.

Device drivers for Windows are written to the Windows Driver Model (WDM) specification. This model lays out all the requirements for device drivers, including how to layer filter drivers, share common code for han-

dling power and plug-and-play requests, build correct cancellation logic, and so forth.

Because of the richness of the WDM, writing a full WDM device driver for each new hardware device can involve a great deal of work. In some cases, the port/miniport model makes it unnecessary to do this for certain hardware devices. Within a range of devices that require similar processing, such as audio drivers, storage controllers, or Ethernet controllers, each instance of a device shares a common driver for that class, called a **port driver**. The port driver implements the standard operations for the class and then calls device-specific routines in the device's **miniport driver** to implement device-specific functionality. The physical-link layer of the network stack is implemented in this way, with the `ndis.sys` port driver implementing much of the generic network processing functionality and calling out to the network miniport drivers for specific hardware commands related to sending and receiving network frames (such as Ethernet).

Similarly, the WDM includes a class/miniclass model. Here, a certain class of devices can be implemented in a generic way by a single class driver, with callouts to a miniclass for specific hardware functionality. For example, the Windows disk driver is a class driver, as are drivers for CD/DVDs and tape drives. The keyboard and mouse driver are class drivers as well. These types of devices don't need a miniclass, but the battery class driver, for example, does require a miniclass for each of the various external uninterruptible power supplies (UPSs) sold by vendors.

Even with the port/miniport and class/miniclass model, significant kernel-facing code must be written. And this model is not useful for custom hardware or for logical (nonhardware) drivers. Starting with Windows 2000 Service Pack 4, kernel-mode drivers can be written using the **Kernel-Mode Driver Framework** (**KMDF**), which provides a simplified programming model for drivers on top of WDM. Another option is the **User-Mode Driver Framework** (**UMDF**), which allows drivers to be written in user mode through a **reflecto** driver in the kernel that forwards the requests through the kernel's I/O stack. These two frameworks make up the **Windows Driver Foundation** model, which has reached Version 2.1 in Windows 10 and contains a fully compatible API between KMDF and UMDF. It has been fully open-sourced on GitHub.

Because many drivers do not need to operate in kernel mode, and it is easier to develop and deploy drivers in user mode, UMDF is strongly recommended for new drivers. It also makes the system more reliable, because a failure in a user-mode driver does not cause a kernel (system) crash.

21.3.5.6 Cache Manager

In many operating systems, caching is done by the block device system, usually at the physical/block level. Instead, Windows provides a centralized caching facility that operates at the logical/virtual file level. The **cache manager** works closely with the MM to provide cache services for all components under the control of the I/O manager. This means that the cache can operate on anything from remote files on a network share to logical files on a custom file system. The size of the cache changes dynamically according to how much free memory is available in the system; it can grow as large as 2 TB on a 64-bit system.

The cache manager maintains a private working set rather than sharing the system process's working set, which allows trimming to page out cached files more effectively. To build the cache, the cache manager memory-maps files into kernel memory and then uses special interfaces to the MM to fault pages into or trim them from this private working set, which lets it take advantage of additional caching facilities provided by the memory manager.

The cache is divided into blocks of 256 KB. Each cache block can hold a view (that is, a memory-mapped region) of a file. Each cache block is described by a **virtual address control block** (**VACB**) that stores the virtual address and file offset for the view, as well as the number of processes using the view. The VACBs reside in arrays maintained by the cache manager, and there are arrays for critical as well as low-priority cached data to improve performance in situations of memory pressure.

When the I/O manager receives a file's user-level read request, the I/O manager sends an IRP to the I/O stack for the volume on which the file resides. For files that are marked as cacheable, the file system calls the cache manager to look up the requested data in its cached file views. The cache manager calculates which entry of that file's VACB index array corresponds to the byte offset of the request. The entry either points to the view in the cache or is invalid. If it is invalid, the cache manager allocates a cache block (and the corresponding entry in the VACB array) and maps the view into the cache block. The cache manager then attempts to copy data from the mapped file to the caller's buffer. If the copy succeeds, the operation is completed.

If the copy fails, it does so because of a page fault, which causes the MM to send a noncached read request to the I/O manager. The I/O manager sends another request down the driver stack, this time requesting a paging operation, which bypasses the cache manager and reads the data from the file directly into the page allocated for the cache manager. Upon completion, the VACB is set to point at the page. The data, now in the cache, are copied to the caller's buffer, and the original I/O request is completed. Figure 21.6 shows an overview of these operations.

When possible, for synchronous operations on cached files, I/O is handled by the **fast I/O mechanism**. This mechanism parallels the normal IRP-based I/O

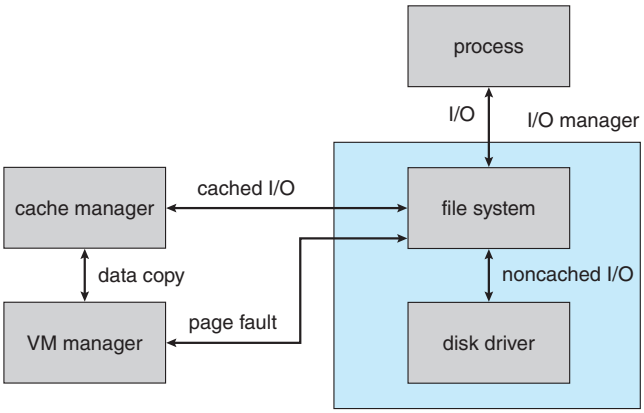


Figure 21.6 File I/O.

but calls into the driver stack directly rather than passing down an IRP, which saves memory and time. Because no IRP is involved, the operation should not block for an extended period of time and cannot be queued to a worker thread. Therefore, when the operation reaches the file system and calls the cache manager, the operation fails if the information is not already in the cache. The I/O manager then attempts the operation using the normal IRP path.

A kernel-level read operation is similar, except that the data can be accessed directly from the cache rather than being copied to a buffer in user space. To use file-system metadata (data structures that describe the file system), the kernel uses the cache manager's mapping interface to read the metadata. To modify the metadata, the file system uses the cache manager's pinning interface. **Pinning** a page locks the page into a physical-memory page frame so that the MM manager cannot move the page or page it out. After updating the metadata, the file system asks the cache manager to unpin the page. A modified page is marked dirty, and so the MM flushes the page to secondary storage.

To improve performance, the cache manager keeps a small history of read requests and from this history attempts to predict future requests. If the cache manager finds a pattern in the previous three requests, such as sequential access forward or backward, it prefetches data into the cache before the next request is submitted by the application. In this way, the application may find its data already cached and not need to wait for secondary storage I/O.

The cache manager is also responsible for telling the MM to flush the contents of the cache. The cache manager's default behavior is write-back caching: it accumulates writes for 4 to 5 seconds and then wakes up the cache-writer thread. When write-through caching is needed, a process can set a flag when opening the file, or can call an explicit cache-flush function.

A fast-writing process could potentially fill all the free cache pages before the cache-writer thread had a chance to wake up and flush the pages to secondary storage. The cache writer prevents a process from flooding the system in the following way. When the amount of free cache memory becomes low, the cache manager temporarily blocks processes attempting to write data and wakes the cache-writer thread to flush pages to secondary storage. If the fast-writing process is actually a network redirector for a network file system, blocking it for too long could cause network transfers to time out and be retransmitted. This retransmission would waste network bandwidth. To prevent such waste, network redirectors can instruct the cache manager to limit the backlog of writes in the cache.

Because a network file system needs to move data between secondary storage and the network interface, the cache manager also provides a DMA interface to move the data directly. Moving data directly avoids the need to copy data through an intermediate buffer.

21.3.5.7 Security Reference Monitor

Centralizing management of system entities in the object manager enables Windows to use a uniform mechanism to perform run-time access validation and audit checks for every user-accessible entity in the system. Additionally, even entities not managed by the object manager may have access to the API routines for performing security checks. Whenever a thread opens a handle to a protected data structure (such as an object), the **security reference monitor**

(**SRM**) checks the effective security token and the object's security descriptor, which contains two access-control lists—the discretionary access control list (DACL) and the system access control list (SACL)—to see whether the process has the necessary access rights. The effective security token is typically the token of the thread's process, but it can also be the token of the thread itself, as described below.

Each process has an associated **security token**. When the login process (`lsass.exe`) authenticates a user, the security token is attached to the user's first process (`userinit.exe`) and copied for each of its child processes. The token contains the **security identity** (**SID**) of the user, the SIDs of the groups the user belongs to, the privileges the user has, the integrity level of the process, the attributes and claims associated with the user, and any relevant capabilities. By default, threads don't have their own explicit tokens, causing them to share the common token of the process. However, using a mechanism called **impersonation**, a thread running in a process with a security token belonging to one user can set a thread-specific token belonging to another user to impersonate that user. At this point, the effective token becomes the token of the thread, and all operations, quotas, and limitations are subject to that user's token. The thread can later choose to "revert" to its old identity by removing the thread-specific token, so that the effective token is once again that of the process.

This impersonation facility is fundamental to the client–server model, where services must act on behalf of a variety of clients with different security IDs. The right to impersonate a user is most often delivered as part of a connection from a client process to a server process. Impersonation allows the server to access system services as if it were the client in order to access or create objects and files on behalf of the client. The server process must be trustworthy and must be carefully written to be robust against attacks. Otherwise, one client could take over a server process and then impersonate any user who made a subsequent client request. Windows provides APIs to support impersonation at the ALPC (and thus RPC and DCOM) layer, the named pipe layer, and the Winsock layer.

The SRM is also responsible for manipulating the privileges in security tokens. Special privileges are required for users to change the system time, load a driver, or change firmware environment variables. Additionally, certain users can have powerful privileges that override default access control rules. These include users who must perform backup or restore operations on file systems (allowing them to bypass read/write restrictions), debug processes (allowing them to bypass security features), and so forth.

The integrity level of the code executing in a process is also represented by a token. Integrity levels are a type of mandatory labeling mechanism, as mentioned earlier. By default, a process cannot modify an object with an integrity level higher than that of the code executing in the process, whatever other permissions have been granted. In addition, it cannot read from another process object at a higher integrity level. Objects can also protect themselves from read access by manually changing the mandatory policy associated with their security descriptor. Inside an object (such as a file or a process), the integrity level is stored in the SACL, which distinguishes it from typical discretionary user and group permissions, stored in the DACL.

Integrity levels were introduced to make it harder for code to take over a system by attacking external-content-parsing software, like a browser or

PDF reader, because such software is expected to run at a low integrity level. For example, Microsoft Edge runs at "low integrity," as do Adobe Reader and Google Chrome. A regular application, such as Microsoft Word, runs at "medium integrity." Finally, you can expect an application run by an administrator or a setup program to run at "high integrity."

Creating applications to run at lower integrity levels places a burden on the developers to implement this security feature, because they must create a client–server model to support a broker and parser or renderer, as mentioned earlier. In order to streamline this security model, Windows 8 introduced the **Application Container**, often just called "AppContainer," which is a special extension of the token object. When running under an AppContainer, an application automatically has its process token adjusted in the following ways:

1. The token's integrity level is set to low. This means that the application cannot write to or modify most objects (files, keys, processes) on the system, nor can it read from any other process on the system.
2. All groups and the user SID are disabled (ignored) in the token. Let's say that the application was launched by user Anne, who belongs to the World group. Any files accessible to Anne or World will be inaccessible to this application.
3. All privileges except a handful are removed from the token. This prevents powerful system calls or system-wide operations from being permitted.
4. A special AppContainer SID is added to the token, which corresponds to the SHA-256 hash of the application's package identifier. This is the only valid security identifier in the token, so any object wishing to be directly accessible to this application needs to explicitly give the AppContainer SID read or write access.
5. A set of capability SIDs are added to the token, based on the application's manifest file. When the application is first installed, these capabilities are shown to the user, who must agree to them before the application is deployed.

We can see that the AppContainer mechanism changes the security model from a discretionary system where access to protected resources is defined by users and groups to a mandatory system where each application has its own unique security identity and access occurs on a per-application basis. This separation of privileges and permissions is a great leap forward in security, but it places a potential burden on resource access. Capabilities and brokers help to alleviate this burden.

Capabilities are used by system brokers implemented by Windows to perform various actions on behalf of packaged applications. For example, assume that Harold's packaged application has no access to Harold's file system, since the Harold SID is disabled. In this situation, a broker might check for the `Play User Media` capability and allow the music player process to read any MP3 files located in Harold's `My Music` directory. Thus, Harold will not be forced to mark all of his files with the AppContainer SID of his favorite media player application, as long as the application has the `Play User Media` capability and Harold agreed to it when he downloaded the application.

A final responsibility of the SRM is logging security audit events. The ISO standard **Common Criteria** (the international successor to the Orange Book standard developed by the United States Department of Defense) requires that a secure system have the ability to detect and log all attempts to access system resources so that it can more easily trace attempts at unauthorized access. Because the SRM is responsible for making access checks, it generates most of the audit records, which are then written by `lsass.exe` into the security-event log.

21.3.5.8 Plug-and-Play Manager

The operating system uses the **plug-and-play** (**PnP**) **manager** to recognize and adapt to changes in hardware configuration. PnP devices use standard protocols to identify themselves to the system. The PnP manager automatically recognizes installed devices and detects changes in devices as the system operates. The manager also keeps track of hardware resources used by a device, as well as potential resources that could be used, and takes care of loading the appropriate drivers. This management of hardware resources—primarily interrupts, DMA channels, and I/O memory ranges—has the goal of determining a hardware configuration in which all devices are able to operate successfully. The PnP manager and the Windows Driver Model see drivers as either `bus drivers`, which detect and enumerate the devices on a bus (such as PCI or USB), or `function drivers`, which implement the functionality of a particular device on the bus.

The PnP manager handles dynamic reconfiguration as follows. First, it gets a list of devices from each bus driver. It loads the drivers and sends an `add-device` request to the appropriate driver for each device. Working in tandem with special **resource arbiters** owned by the various bus drivers, the PnP manager then figures out the optimal resource assignments and sends a `start-device` request to each driver specifying the resource assignments for the related devices. If a device needs to be reconfigured, the PnP manager sends a `query-stop` request, which asks the driver whether the device can be temporarily disabled. If the driver can disable the device, then all pending operations are completed, and new operations are prevented from starting. Finally, the PnP manager sends a `stop` request and can then reconfigure the device with a new `start-device` request.

The PnP manager also supports other requests. For example, `query-remove`, which operates similarly to `query-stop`, is employed when a user is getting ready to eject a removable device, such as a USB storage device. The `surprise-remove` request is used when a device fails or, more often, when a user removes a device without telling the system to stop it first. Finally, the `remove` request tells the driver to stop using a device permanently.

Many programs in the system are interested in the addition or removal of devices, so the PnP manager supports notifications. Such a notification, for example, gives the file manager the information it needs to update its list of secondary storage volumes when a new storage device is attached or removed.

Installing devices can also result in starting new services on the system. Previously, such services frequently set themselves up to run whenever the system booted and continued to run even if the associated device was never plugged into the system, because they had to be running in order to receive the

PnP notification. Windows 7 introduced a **service-trigger** mechanism in the **service control manager** (**SCM**) (`services.exe`), which manages the system services. With this mechanism, services can register themselves to start only when SCM receives a notification from the PnP manager that the device of interest has been added to the system.

21.3.5.9 Power Manager

Windows works with the hardware to implement sophisticated strategies for energy efficiency, as described in Section 21.2.8. The policies that drive these strategies are implemented by the **power manager**. The power manager detects current system conditions, such as the load on CPUs or I/O devices, and improves energy efficiency by reducing the performance and responsiveness of the system when need is low. The power manager can also put the entire system into a very efficient ***sleep*** mode and can even write all the contents of memory to secondary storage and turn off the power to allow the system to go into ***hibernation***.

The primary advantage of sleep is that the system can enter that state fairly quickly, perhaps just a few seconds after the lid closes on a laptop. The return from sleep is also fairly quick. The power is turned down to a low level on the CPUs and I/O devices, but the memory continues to be powered enough that its contents are not lost. As noted earlier, however, on mobile devices, these few seconds still add up to an unreasonable user experience, so the power manager works with the Desktop Activity Moderator to kick off the Connected Standby state as soon as the screen is turned off. Connected Standby virtually freezes the computer but does not really put the computer to sleep.

Hibernation takes considerably longer to enter than sleep because the entire contents of memory must be transferred to secondary storage before the system is turned off. However, the fact that the system is, in fact, turned off is a significant advantage. If there is a loss of power to the system, as when the battery is swapped on a laptop or a desktop system is unplugged, the saved system data will not be lost. Unlike shutdown, hibernation saves the currently running system so a user can resume where she left off. Furthermore, because hibernation does not require power, a system can remain in hibernation indefinitely. Therefore, this feature is extremely useful on desktops and server systems, and it is also used on laptops when the battery hits a critical level (because putting the system to sleep when the battery is low might result in the loss of all data if the battery runs out of power while in the sleep state).

In Windows 7, the power manager also includes a processor power manager (PPM), which specifically implements strategies such as core parking, CPU throttling and boosting, and more. In addition, Windows 8 introduced the **power framework** (**PoFX**), which works with function drivers to implement specific functional power states. This means that devices can expose their internal power management (clock speeds, current/power draws, and so forth) to the system, which can then use the information for fine-grained control of the devices. Thus, for example, instead of simply turning a device on or off, the system can turn specific components on or off.

Like the PnP manager, the power manager provides notifications to the rest of the system about changes in the power state. Some applications want to know when the system is about to be shut down so they can start saving their

states to secondary storage, and, as mentioned earlier, the DAM needs to know when the screen is turned off and on again.

21.3.5.10 Registry

Windows keeps much of its configuration information in internal repositories of data, called **hives**, that are managed by the Windows configuration manager, commonly known as the **registry**. The configuration manager is implemented as a component of the executive.

There are separate hives for system information, each user's preferences, software information, security, and boot options. Additionally, as part of the new application and security model introduced by AppContainers and UWPModern/Metro packaged applications in Windows 8, each such application has its own separate hive, called an application hive.

The registry represents the configuration state in each hive as a hierarchical namespace of keys (directories), each of which can contain a set of arbitrarily sized values. In the Win32 API, these values have a specific "type," such as UNICODE string, 32-bit integer, or untyped binary data, but the registry itself treats all values the same, leaving it up to the higher API layers to infer a structure based on type and size. Therefore, for example, nothing prevents a "32-bit integer" from being a 999-byte UNICODE string.

In theory, new keys and values are created and initialized as new software is installed, and then they are modified to reflect changes in the configuration of that software. In practice, the registry is often used as a general-purpose database, as an interprocess-communication mechanism, and for many other such inventive purposes.

Restarting applications, or even the system, every time a configuration change was made would be a nuisance. Instead, programs rely on various kinds of notifications, such as those provided by the PnP and power managers, to learn about changes in the system configuration. The registry also supplies notifications; threads can register to be notified when changes are made to some part of the registry. The threads can thus detect and adapt to configuration changes recorded in the registry. Furthermore, registry keys are objects managed by the object manager, and they expose an event object to the dispatcher. This allows threads to put themselves in a `waiting` state associated with the event, which the configuration manager will signal if the key (or any of its values) is ever modified.

Whenever significant changes are made to the system, such as when updates to the operating system or drivers are installed, there is a danger that the configuration data may be corrupted (for example, if a working driver is replaced by a nonworking driver or an application fails to install correctly and leaves partial information in the registry). Windows creates a **system restore point** before making such changes. The restore point contains a copy of the hives before the change and can be used to return to this version of the hives in order to get a corrupted system working again.

To improve the stability of the registry configuration, the registry also implements a variety of "self-healing" algorithms, which can detect and fix certain cases of registry corruption. Additionally, the registry internally uses a two-phase commit transactional algorithm, which prevents corruption to individual keys or values as they are being updated. While these mechanisms

guarantee the integrity of small portions of the registry or individual keys and values, they have not supplanted the system restore facility for recovering from damage to the registry configuration caused by a failure during a software installation.

21.3.5.11 Booting

The booting of a Windows PC begins when the hardware powers on and firmware begins executing from ROM. In older machines, this firmware was known as the BIOS, but more modern systems use UEFI (the Unified Extensible Firmware Interface), which is faster, is more modern, and makes better use of the facilities in contemporary processors. Additionally, UEFI includes a feature called **Secure Boot** that provides integrity checks through digital signature verification of all firmware and boot-time components. This digital signature check guarantees that only Microsoft's boot-time components and the vendor's firmware are present at boot time, preventing any early third-party code from loading.

The firmware runs **power-on self-test** (**POST**) diagnostics, identifies many of the devices attached to the system and initializes them to a clean power-up state, and then builds the description used by ACPI. Next, the firmware finds the system boot device, loads the Windows boot manager program (`bootmgfw.efi` on UEFI systems), and begins executing it.

In a machine that has been hibernating, the `winresume.efi` program is loaded next. It restores the running system from secondary storage, and the system continues execution at the point it had reached right before hibernating. In a machine that has been shut down, `bootmgfw.efi` performs further initialization of the system and then loads `winload.efi`. This program loads `hal.dll`, the kernel (`ntoskrnl.exe`) and its dependencies, and any drivers needed in booting, and the system hive. `winload` then transfers execution to the kernel.

The procedure is somewhat different on Windows 10 systems where Virtual Secure Mode is enabled (and the hypervisor is turned on). Here, `winload.efi` will instead load `hvloader.exe` or `hvloader.dll`, which initializes the hypervisor first. On Intel systems, this is `hvix64.exe`, while AMD systems use `hvax64.exe`. The hypervisor then sets up VTL 1 (the Secure World) and VTL 0 (the Normal World) and returns to `winload.efi`, which now loads the secure kernel (`securekernel.exe`) and its dependencies. Then the secure kernel's entry point is called, which initializes VTL 1, after which it returns back to the loader at VTL 0, which resumes with the steps described above.

As the kernel initializes itself, it creates several processes. The **idle process** serves as the container of all idle threads, so that system-wide CPU idle time can easily be computed. The **system process** contains all of the internal kernel worker threads and other system threads created by drivers for polling, housekeeping, and other background work. The memory compression process, new to Windows 10, has a working set composed of compressed standby pages used by the store manager to alleviate system pressure and optimize paging. Finally, if VSM is enabled, the **secure system process** represents the fact that the secure kernel is loaded.

The first user-mode process, which is also created by the kernel, is **session manager subsystem** (**SMSS**), which is similar to the `init` (initialization)

process in UNIX. SMSS performs further initialization of the system, including establishing the paging files and creating the initial user sessions. Each session represents a logged-on user, except for ***session 0***, which is used to run system-wide background processes, such as `lsass` and `services`. Each session is given its own instance of an SMSS process, which exits once the session is created. In each of these sessions, this ephemeral SMSS loads the Win32 environment subsystem (`csrss.exe`) and its driver (`win32k.sys`). Then, in each session other than 0, SMSS runs the `winlogon` process, which launches `logonui`. This process captures user credentials in order for `lsass` to log in a user, then launch the `userinit` and `explorer` process, which implements the Windows shell (start menu, desktop, tray icons, notification center, and so on). The following list itemizes some of these aspects of booting:

- SMSS completes system initialization and then starts up one SMSS for session 0 and one SMSS for the first login session (1).
- `wininit` runs in session 0 to initialize user mode and start `lsass` and `services`.
- `lsass`, the security subsystem, implements facilities such as authentication of users. If user credentials are protected by VSMthrough Credential Guard, then `lsaiso` and `bioiso` are also started as VTL 1 Trustlets by `lsass`.
- `services` contains the service control manager, or SCM, which supervises all background activities in the system, including user-mode services. A number of services will have registered to start when the system boots. Others will be started only on demand or when triggered by an event such as the arrival of a device.
- `csrss` is the Win32 environment subsystem process. It is started in every session—mainly because it handles mouse and keyboard input, which needs to be separated per user.
- `winlogon` is run in each Windows session other than session 0 to log on a user by launching `logonui`, which presents the logon user interface.

Starting with Windows XP, the system optimizes the boot process by prefetching pages from files on secondary storage based on previous boots of the system. Disk access patterns at boot are also used to lay out system files on disk to reduce the number of I/O operations required. Windows 7 reduced the processes necessary to start the system by allowing services to start only when needed, rather than at system start-up. Windows 8 further reduced boot time by parallelizing all driver loads through a pool of worker threads in the PnP subsystem and by supporting UEFI to make boot-time transition more efficient. All of these approaches contributed to a dramatic reduction in system boot time, but eventually little further improvement was possible.

To address boot-time concerns, especially on mobile systems, where RAM and cores are limited, Windows 8 also introduced **Hybrid Boot**. This feature combines hibernation with a simple logoff of the current user. When the user shuts down the system, and all other applications and sessions have exited, the system is returned to the `logonui` prompt and then is hibernated. When the system is turned on again, it resumes very quickly to the logon screen, which

gives drivers a chance to reinitialize devices and gives the appearance of a full boot while work is still occurring.

21.4 Terminal Services and Fast User Switching

Windows supports a GUI-based console that interfaces with the user via keyboard, mouse, and display. Most systems also support audio and video. For example, audio input is used by Cortana, Windows's voice-recognition and virtual assistant software, which is powered by machine learning. Cortana makes the system more convenient and can also increase its accessibility for users with motor disabilities. Windows 7 added support for **multi-touch hardware**, allowing users to input data by touching the screen with one or more fingers. Video-input capability is used both for accessibility and for security: Windows Hello is a security feature in which advanced 3D heat-sensing, face-mapping cameras and sensors can be used to uniquely identify the user without requiring traditional credentials. In newer versions of Windows 10, eye-motion sensing hardware—in which mouse input is replaced by information on the position and gaze of the eyeballs—can be used for accessibility. Other future input experiences will likely evolve from Microsoft's **HoloLens** augmented-reality product.

The PC was, of course, envisioned as a ***personal computer***—an inherently single-user machine. For some time, however, Windows has supported the sharing of a PC among multiple users. Each user who is logged on using the GUI has a **session** created to represent the GUI environment he will be using and to contain all the processes necessary to run his applications. Windows allows multiple sessions to exist at the same time on a single machine. However, client versions of Windows support only a single console, consisting of all the monitors, keyboards, and mice connected to the PC. Only one session can be connected to the console at a time. From the logon screen displayed on the console, users can create new sessions or attach to an existing session. This allows multiple users to share a single PC without having to log off and on between users. Microsoft calls this use of sessions ***fast user switching.*** macOS has a similar feature.

A user on one PC can also create a new session or connect to an existing session on another computer, which becomes a **remote desktop**. The terminal services feature (TS) makes the connection through a protocol called Remote Desktop Protocol (RDP). Users often employ this feature to connect to a session on a work PC from a home PC. Remote desktops can also be used for remote troubleshooting scenarios: a remote user can be invited to share a session with the user logged on to the session on the console. The remote user can watch the user's actions and can even be given control of the desktop to help resolve computing problems. This latter use of terminal services uses the "mirroring" feature, where the alternative user is sharing the same session instead of creating a separate one.

Many corporations use corporate systems maintained in data centers to run all user sessions that access corporate resources, rather than allowing users to access those resources from their PCs, by exclusively dedicating these machines as terminal servers. Each server computer may handle hundreds of remote-desktop sessions. This is a form of **thin-client computing**, in which

individual computers rely on a server for many functions. Relying on data-center terminal servers improves the reliability, manageability, and security of corporate computing resources.

21.5 File System

The native file system in Windows is NTFS. It is used for all local volumes. However, associated USB thumb drives, flash memory on cameras, and external storage devices may be formatted with the 32-bit FAT file system for portability. FAT is a much older file-system format that is understood by many systems besides Windows, such as the software running on cameras. A disadvantage is that the FAT file system does not restrict file access to authorized users. The only solution for securing data with FAT is to run an application to encrypt the data before storing it on the file system.

In contrast, NTFS uses ACLs to control access to individual files and supports implicit encryption of individual files or entire volumes (using Windows BitLocker feature). NTFS implements many other features as well, including data recovery, fault tolerance, very large files and file systems, multiple data streams, UNICODE names, sparse files, journaling, volume shadow copies, and file compression.

21.5.1 NTFS Internal Layout

The fundamental entity in NTFS is the volume. A volume is created by the Windows logical disk management utility and is based on a logical disk partition. A volume may occupy a portion of a device or an entire device, or may span several devices. The volume manager can protect the contents of the volume with various levels of RAID.

NTFS does not deal with individual sectors of a storage device but instead uses **clusters** as the units of storage allocation. The cluster size, which is a power of 2, is configured when an NTFS file system is formatted. The default cluster size is based on the volume size—4 KB for volumes larger than 2 GB. Given the size of today's storage devices, it may make sense to use cluster sizes larger than the Windows defaults to achieve better performance, although these performance gains will come at the expense of more internal fragmentation.

NTFS uses **logical cluster numbers** (**LCNs**) as storage addresses. It assigns them by numbering clusters from the beginning of the device to the end. Using this scheme, the system can calculate a physical storage offset (in bytes) by multiplying the LCN by the cluster size.

A file in NTFS is not a simple byte stream as it is in UNIX; rather, it is a structured object consisting of typed **attributes**. Each attribute of a file is an independent byte stream that can be created, deleted, read, and written. Some attribute types are standard for all files, including the file name (or names, if the file has aliases, such as an MS-DOS short name), the creation time, and the security descriptor that specifies the access control list. User data are stored in ***data attributes***.

Most traditional data files have an ***unnamed*** data attribute that contains all the file's data. However, additional data streams can be created with explicit names. The IProp interfaces of the Component Object Model (discussed later

in this chapter) use a named data stream to store properties on ordinary files, including thumbnails of images. In general, attributes can be added as necessary and are accessed using a ***file-name:attribute*** syntax. NTFS returns only the size of the unnamed attribute in response to file-query operations, such as when running the `dir` command.

Every file in NTFS is described by one or more records in an array stored in a special file called the master file table (MFT). The size of a record is determined when the file system is created; it ranges from 1 to 4 KB. Small attributes are stored in the MFT record itself and are called ***resident attributes***. Large attributes, such as the unnamed bulk data, are called ***nonresident attributes*** and are stored in one or more contiguous extents on the device. A pointer to each extent is stored in the MFT record. For a small file, even the data attribute may fit inside the MFT record. If a file has many attributes—or if it is highly fragmented, so that many pointers are needed to point to all the fragments—one record in the MFT might not be large enough. In this case, the file is described by a record called the **base file record**, which contains pointers to overflow records that hold the additional pointers and attributes.

Each file in an NTFS volume has a unique ID called a **file reference**. The file reference is a 64-bit quantity that consists of a 48-bit file number and a 16-bit sequence number. The file number is the record number (that is, the array slot) in the MFT that describes the file. The sequence number is incremented every time an MFT entry is reused. The sequence number enables NTFS to perform internal consistency checks, such as catching a stale reference to a deleted file after the MFT entry has been reused for a new file.

21.5.1.1 NTFS B+ Tree

As in UNIX, the NTFS namespace is organized as a hierarchy of directories. Each directory uses a data structure called a **B+ tree** to store an index of the file names in that directory. In a B+ tree, the length of every path from the root of the tree to a leaf is the same, and the cost of reorganizing the tree is eliminated. The **index root** of a directory contains the top level of the B+ tree. For a large directory, this top level contains pointers to disk extents that hold the remainder of the tree. Each entry in the directory contains the name and file reference of the file, as well as a copy of the update timestamp and file size taken from the file's resident attributes in the MFT. Copies of this information are stored in the directory so that a directory listing can be efficiently generated. Because all the file names, sizes, and update times are available from the directory itself, there is no need to gather these attributes from the MFT entries for each of the files.

21.5.1.2 NTFS Metadata

The NTFS volume's metadata are all stored in files. The first file is the MFT. The second file, which is used during recovery if the MFT is damaged, contains a copy of the first 16 entries of the MFT. The next few files are also special in purpose. They include the following files:

- The **log file** records all metadata updates to the file system.
- The **volume file** contains the name of the volume, the version of NTFS that formatted the volume, and a bit that tells whether the volume may have

been corrupted and needs to be checked for consistency using the `chkdsk` program.

- The **attribute-definitio table** indicates which attribute types are used in the volume and what operations can be performed on each of them.
- The **root directory** is the top-level directory in the file-system hierarchy.
- The **bitmap fil** indicates which clusters on a volume are allocated to files and which are free.
- The **boot fil** contains the startup code for Windows and must be located at a particular secondary storage device address so that it can be found easily by a simple ROM bootstrap loader. The boot file also contains the physical address of the MFT.
- The **bad-cluster file** keeps track of any bad areas on the volume; NTFS uses this record for error recovery.

Keeping all the NTFS metadata in actual files has a useful property. As discussed in Section 21.3.5.6, the cache manager caches file data. Since all the NTFS metadata reside in files, these data can be cached using the same mechanisms used for ordinary data.

21.5.2 Recovery

In many simple file systems, a power failure at the wrong time can damage the file-system data structures so severely that the entire volume is scrambled. Many UNIX file systems, including UFS but not ZFS, store redundant metadata on the storage device, and they recover from crashes by using the `fsck` program to check all the file-system data structures and restore them forcibly to a consistent state. Restoring them often involves deleting damaged files and freeing data clusters that had been written with user data but not properly recorded in the file system's metadata structures. This checking can be a slow process and can result in the loss of significant amounts of data.

NTFS takes a different approach to file-system robustness. In NTFS, all file-system data-structure updates are performed inside transactions. Before a data structure is altered, the transaction writes a log record that contains redo and undo information. After the data structure has been changed, the transaction writes a commit record to the log to signify that the transaction succeeded.

After a crash, the system can restore the file-system data structures to a consistent state by processing the log records, first redoing the operations for committed transactions (to be sure their changes reached the file system data structures) and then undoing the operations for transactions that did not commit successfully before the crash. Periodically (usually every 5 seconds), a checkpoint record is written to the log. The system does not need log records prior to the checkpoint to recover from a crash. They can be discarded, so the log file does not grow without bounds. The first time after system startup that an NTFS volume is accessed, NTFS automatically performs file-system recovery.

This scheme does not guarantee that all the user-file contents are correct after a crash. It ensures only that the file-system data structures (the metadata files) are undamaged and reflect some consistent state that existed prior to the

crash. It would be possible to extend the transaction scheme to cover user files, and Microsoft took some steps to do this in Windows Vista.

The log is stored in the third metadata file at the beginning of the volume. It is created with a fixed maximum size when the file system is formatted. It has two sections: the ***logging area,*** which is a circular queue of log records, and the ***restart area,*** which holds context information, such as the position in the logging area where NTFS should start reading during a recovery. In fact, the restart area holds two copies of its information, so recovery is still possible if one copy is damaged during the crash.

The logging functionality is provided by the ***log-file service.*** In addition to writing the log records and performing recovery actions, the log-file service keeps track of the free space in the log file. If the free space gets too low, the log-file service queues pending transactions, and NTFS halts all new I/O operations. After the in-progress operations complete, NTFS calls the cache manager to flush all data and then resets the log file and performs the queued transactions.

21.5.3 Security

The security of an NTFS volume is derived from the Windows object model. Each NTFS file references a security descriptor, which specifies the owner of the file, and an access-control list, which contains the access permissions granted or denied to each user or group listed. Early versions of NTFS used a separate security descriptor as an attribute of each file. Beginning with Windows 2000, the security-descriptor attribute points to a shared copy, with a significant savings in storage space and caching space; many, many files have identical security descriptors.

In normal operation, NTFS does not enforce permissions on traversal of directories in file path names. However, for compatibility with POSIX, these checks can be enabled. The latter option is inherently more expensive, since modern parsing of file path names uses prefix matching rather than directory-by-directory parsing of path names. Prefix matching is an algorithm that looks up strings in a cache and finds the entry with the longest match—for example, an entry for `\foo\bar\dir` would be a match for `\foo\bar\dir2\dir3\myfile`. The prefix-matching cache allows path-name traversal to begin much deeper in the tree, saving many steps. Enforcing traversal checks means that the user's access must be checked at each directory level. For instance, a user might lack permission to traverse `\foo\bar`, so starting at the access for `\foo\bar\dir` would be an error.

21.5.4 Compression

NTFS can perform data compression on individual files or on all data files in a directory. To compress a file, NTFS divides the file's data into **compression units**, which are blocks of 16 contiguous clusters. When a compression unit is written, a data-compression algorithm is applied. If the result fits into fewer than 16 clusters, the compressed version is stored. When reading, NTFS can determine whether data have been compressed: if they have been, the length of the stored compression unit is less than 16 clusters. To improve performance when reading contiguous compression units, NTFS prefetches and decompresses ahead of the application requests.

For sparse files or files that contain mostly zeros, NTFS uses another technique to save space. Clusters that contain only zeros because they have never been written are not actually allocated or stored on storage devices. Instead, gaps are left in the sequence of virtual-cluster numbers stored in the MFT entry for the file. When reading a file, if NTFS finds a gap in the virtual-cluster numbers, it just zero-fills that portion of the caller's buffer. This technique is also used by UNIX.

21.5.5 Mount Points, Symbolic Links, and Hard Links

Mount points are a form of symbolic link specific to directories on NTFS that were introduced in Windows 2000. They provide a mechanism for organizing storage volumes that is more flexible than the use of global names (like drive letters). A mount point is implemented as a symbolic link with associated data containing the true volume name. Ultimately, mount points will supplant drive letters completely, but there will be a long transition due to the dependence of many applications on the drive-letter scheme.

Windows Vista introduced support for a more general form of symbolic links, similar to those found in UNIX. The links can be absolute or relative, can point to objects that do not exist, and can point to both files and directories even across volumes. NTFS also supports **hard links**, where a single file has an entry in more than one directory of the same volume.

21.5.6 Change Journal

NTFS keeps a journal describing all changes that have been made to the file system. User-mode services can receive notifications of changes to the journal and then identify what files have changed by reading from the journal. The search indexer service uses the change journal to identify files that need to be re-indexed. The file-replication service uses it to identify files that need to be replicated across the network.

21.5.7 Volume Shadow Copies

Windows implements the capability of bringing a volume to a known state and then creating a shadow copy that can be used to back up a consistent view of the volume. This technique is known as ***snapshots*** in some other file systems. Making a shadow copy of a volume is a form of copy-on-write, where blocks modified after the shadow copy is created are stored in their original form in the copy. Achieving a consistent state for the volume requires the cooperation of applications, since the system cannot know when the data used by the application are in a stable state from which the application could be safely restarted.

The server version of Windows uses shadow copies to efficiently maintain old versions of files stored on file servers. This allows users to see documents as they existed at earlier points in time. A user can thus recover files that were accidentally deleted or simply look at a previous version of the file, all without pulling out backup media.

21.6 Networking

Windows supports both peer-to-peer and client–server networking. It also has facilities for network management. The networking components in Windows provide data transport, interprocess communication, file sharing across a network, and the ability to send print jobs to remote printers.

21.6.1 Network Interfaces

To describe networking in Windows, we must first mention two of the internal networking interfaces: the **Network Device Interface specificatio** (**NDIS**) and the **Transport Driver Interface** (**TDI**). The NDIS interface was developed in 1989 by Microsoft and 3Com to separate network adapters from transport protocols so that either could be changed without affecting the other. NDIS resides at the interface between the data-link and network layers in the ISO model and enables many protocols to operate over many different network adapters. In terms of the ISO model, the TDI is the interface between the transport layer (layer 4) and the session layer (layer 5). This interface enables any session-layer component to use any available transport mechanism. (Similar reasoning led to the streams mechanism in UNIX.) The TDI supports both connection-based and connectionless transport and has functions to send any type of data.

21.6.2 Protocols

Windows implements transport protocols as drivers. These drivers can be loaded and unloaded from the system dynamically, although in practice the system typically has to be rebooted after a change. Windows comes with several networking protocols. Next, we discuss a number of these protocols.

21.6.2.1 Server Message Block

The **Server Message Block** (**SMB**) protocol was first introduced in MS-DOS 3.1. The system uses the protocol to send I/O requests over the network. The SMB protocol has four message types. `Session control` messages are commands that start and end a redirector connection to a shared resource at the server. A redirector uses `File` messages to access files at the server. `Printer` messages are used to send data to a remote print queue and to receive status information from the queue, and `Message` messages are used to communicate with another workstation. A version of the SMB protocol was published as the **Common Internet File System** (**CIFS**) and is supported on a number of operating systems.

21.6.2.2 Transmission Control Protocol/Internet Protocol

The transmission control protocol/Internet protocol (TCP/IP) suite that is used on the Internet has become the de facto standard networking infrastructure. Windows uses TCP/IP to connect to a wide variety of operating systems and hardware platforms. The Windows TCP/IP package includes the simple network-management protocol (SNMP), the dynamic host-configuration protocol (DHCP), and the older Windows Internet name service (WINS). Windows Vista introduced a new implementation of TCP/IP that supports both `IPv4` and `IPv6` in the same network stack. This new implementation also supports

offloading of the network stack onto advanced hardware to achieve very high performance for servers.

Windows provides a software firewall that limits the TCP ports that can be used by programs for network communication. Network firewalls are commonly implemented in routers and are a very important security measure. Having a firewall built into the operating system makes a hardware router unnecessary, and it also provides more integrated management and easier use.

21.6.2.3 Point-to-Point Tunneling Protocol

The **Point-to-Point Tunneling Protocol** (**PPTP**) is a protocol provided by Windows to communicate between remote-access server modules running on Windows server machines and other client systems that are connected over the Internet. The remote-access servers can encrypt data sent over the connection, and they support multiprotocol **virtual private networks** (**VPNs**) on the Internet.

21.6.2.4 HTTP Protocol

The HTTP protocol is used to `get/put` information using the World Wide Web. Windows implements HTTP using a kernel-mode driver, so web servers can operate with a low-overhead connection to the networking stack. HTTP is a fairly general protocol that Windows makes available as a transport option for implementing RPC.

21.6.2.5 Web-Distributed Authoring and Versioning Protocol

Web-distributed authoring and versioning (WebDAV) is an HTTP-based protocol for collaborative authoring across a network. Windows builds a WebDAV redirector into the file system. Being built directly into the file system enables WebDAV to work with other file-system features, such as encryption. Personal files can then be stored securely in a public place. Because WebDAV uses HTTP, which is a `get/put` protocol, Windows has to cache the files locally so programs can use `read` and `write` operations on parts of the files.

21.6.2.6 Named Pipes

Named pipes are a connection-oriented messaging mechanism. A process can use named pipes to communicate with other processes on the same machine. Since named pipes are accessed through the file-system interface, the security mechanisms used for file objects also apply to named pipes. The SMB protocol supports named pipes, so named pipes can also be used for communication between processes on different systems.

The format of pipe names follows the **Uniform Naming Convention** (**UNC**). A UNC name looks like a typical remote file name. The format is `\\server_name\share_name\x\y\z`, where `server_name` identifies a server on the network; `share_name` identifies any resource that is made available to network users, such as directories, files, named pipes, and printers; and `\x\y\z` is a normal file path name.

21.6.2.7 Remote Procedure Calls

Remote procedure calls (RPCs), mentioned earlier, are client–server mechanisms that enable an application on one machine to make a procedure call to code on another machine. The client calls a local procedure—a stub routine—which packs its arguments into a message and sends them across the network to a particular server process. The client-side stub routine then blocks. Meanwhile, the server unpacks the message, calls the procedure, packs the return results into a message, and sends them back to the client stub. The client stub unblocks, receives the message, unpacks the results of the RPC, and returns them to the caller. This packing of arguments is sometimes called **marshaling**. The client stub code and the descriptors necessary to pack and unpack the arguments for an RPC are compiled from a specification written in the **Microsoft Interface Definitio Language**.

The Windows RPC mechanism follows the widely used distributed-computing-environment standard for RPC messages, so programs written to use Windows RPCs are highly portable. The RPC standard is detailed. It hides many of the architectural differences among computers, such as the sizes of binary numbers and the order of bytes and bits in computer words, by specifying standard data formats for RPC messages.

21.6.2.8 Component Object Model

The **Component Object Model** (**COM**) is a mechanism for interprocess communication that was developed for Windows. A COM object provides a well-defined interface to manipulate the data in the object. For instance, COM is the infrastructure used by Microsoft's **Object Linking and Embedding** (**OLE**) technology for inserting spreadsheets into Microsoft Word documents. Many Windows services provide COM interfaces. In addition, a distributed extension called **DCOM** can be used over a network utilizing RPC to provide a transparent method of developing distributed applications.

21.6.3 Redirectors and Servers

In Windows, an application can use the Windows I/O API to access files from a remote computer as though they were local, provided that the remote computer is running a CIFS server such as those provided by Windows. A **redirector** is the client-side object that forwards I/O requests to a remote system, where they are satisfied by a server. For performance and security, the redirectors and servers run in kernel mode.

In more detail, access to a remote file occurs as follows:

1. The application calls the I/O manager to request that a file be opened with a file name in the standard UNC format.
2. The I/O manager builds an I/O request packet, as described in Section 21.3.5.5.
3. The I/O manager recognizes that the access is for a remote file and calls a driver called a **Multiple UNC Provider** (**MUP**).

4. The MUP sends the I/O request packet asynchronously to all registered redirectors.
5. A redirector that can satisfy the request responds to the MUP. To avoid asking all the redirectors the same question in the future, the MUP uses a cache to remember which redirector can handle this file.
6. The redirector sends the network request to the remote system.
7. The remote-system network drivers receive the request and pass it to the server driver.
8. The server driver hands the request to the proper local file-system driver.
9. The proper device driver is called to access the data.
10. The results are returned to the server driver, which sends the data back to the requesting redirector. The redirector then returns the data to the calling application via the I/O manager.

A similar process occurs for applications that use the Win32 network API, rather than the UNC services, except that a module called a ***multi-provider router*** is used instead of a MUP.

For portability, redirectors and servers use the TDI API for network transport. The requests themselves are expressed in a higher-level protocol, which by default is the SMB protocol described in Section 21.6.2. The list of redirectors is maintained in the system hive of the registry.

21.6.3.1 Distributed File System

UNC names are not always convenient, because multiple file servers may be available to serve the same content and UNC names explicitly include the name of the server. Windows supports a **distributed file-syste** (**DFS**) protocol that allows a network administrator to serve up files from multiple servers using a single distributed name space.

21.6.3.2 Folder Redirection and Client-Side Caching

To improve the PC experience for users who frequently switch among computers, Windows allows administrators to give users **roaming profile** , which keep users' preferences and other settings on servers. **Folder redirection** is then used to automatically store a user's documents and other files on a server.

This works well until one of the computers is no longer attached to the network, as when a user takes a laptop onto an airplane. To give users off-line access to their redirected files, Windows uses **client-side caching** (**CSC**). CSC is also used when the computer is on-line to keep copies of the server files on the local machine for better performance. The files are pushed up to the server as they are changed. If the computer becomes disconnected, the files are still available, and the update of the server is deferred until the next time the computer is online.

21.6.4 Domains

Many networked environments have natural groups of users, such as students in a computer laboratory at school or employees in one department in a business. Frequently, we want all the members of the group to be able to access shared resources on their various computers in the group. To manage the global access rights within such groups, Windows uses the concept of a domain. Previously, these domains had no relationship whatsoever to the domain-name system (DNS) that maps Internet host names to IP addresses. Now, however, they are closely related.

Specifically, a Windows domain is a group of Windows workstations and servers that share a common security policy and user database. Since Windows uses the Kerberos protocol for trust and authentication, a Windows domain is the same thing as a Kerberos realm. Windows uses a hierarchical approach for establishing trust relationships between related domains. The trust relationships are based on DNS and allow transitive trusts that can flow up and down the hierarchy. This approach reduces the number of trusts required for n domains from $n * (n - 1)$ to $O(n)$. The workstations in the domain trust the domain controller to give correct information about the access rights of each user (loaded into the user's access token by `lsaas`). All users retain the ability to restrict access to their own workstations, however, no matter what any domain controller may say.

21.6.5 Active Directory

Active Directory is the Windows implementation of **Lightweight Directory-Access Protocol** (**LDAP**) services. Active Directory stores the topology information about the domain, keeps the domain-based user and group accounts and passwords, and provides a domain-based store for Windows features that need it, such as **Windows group policy**. Administrators use group policies to establish uniform standards for desktop preferences and software. For many corporate information-technology groups, uniformity drastically reduces the cost of computing.

21.7 Programmer Interface

The **Win32 API** is the fundamental interface to the capabilities of Windows. This section describes five main aspects of the Win32 API: access to kernel objects, sharing of objects between processes, process management, interprocess communication, and memory management.

21.7.1 Access to Kernel Objects

The Windows kernel provides many services that application programs can use. Application programs obtain these services by manipulating kernel objects. A process gains access to a kernel object named XXX by calling the `CreateXXX` function to open a handle to an instance of XXX. This handle is unique to the process. Depending on which object is being opened, if the `Create()` function fails, it may return 0, or it may return a special constant named `INVALID_HANDLE_VALUE`. A process can close any handle by calling the

```
SECURITY_ATTRIBUTES sa;
sa.nlength = sizeof(sa);
sa.lpSecurityDescriptor = NULL;
sa.bInheritHandle = TRUE;
HANDLE hSemaphore = CreateSemaphore(&sa, 1, 1, NULL);
WCHAR wszCommandline[MAX_PATH];
StringCchPrintf(wszCommandLine, _countof(wszCommandLine),
    L"another_process.exe %d", hSemaphore);
CreateProcess(L"another_process.exe", wszCommandline,
    NULL, NULL, TRUE, . . .);
```

Figure 21.7 Code enabling a child to share an object by inheriting a handle.

`CloseHandle()` function, and the system may delete the object if the count of handles referencing the object in all processes drops to zero.

21.7.2 Sharing Objects Between Processes

Windows provides three ways to share objects between processes. The first way is for a child process to inherit a handle to the object. When the parent calls the `CreateXXX` function, the parent supplies a `SECURITIES_ATTRIBUTES` structure with the `bInheritHandle` field set to `TRUE`. This field creates an inheritable handle. Next, the child process is created, passing a value of `TRUE` to the `CreateProcess()` function's `bInheritHandle` argument. Figure 21.7 shows a code sample that creates a semaphore handle inherited by a child process.

Assuming the child process knows which handles are shared, the parent and child can achieve interprocess communication through the shared objects. In the example in Figure 21.7, the child process gets the value of the handle from the first command-line argument and then shares the semaphore with the parent process.

The second way to share objects is for one process to give the object a name when the object is created and for the second process to open the name. This method has two drawbacks: Windows does not provide a way to check whether an object with the chosen name already exists, and the object name space is global, without regard to the object type. For instance, two applications may create and share a single object named "foo" when two distinct objects—possibly of different types—were desired.

Named objects have the advantage that unrelated processes can readily share them. The first process calls one of the `CreateXXX` functions and supplies a name as a parameter. The second process gets a handle to share the object by calling `OpenXXX()` (or `CreateXXX`) with the same name, as shown in the example in Figure 21.8.

The third way to share objects is via the `DuplicateHandle()` function. This method requires some other method of interprocess communication to pass the duplicated handle. Given a handle to a process and the value of a handle within that process, a second process can get a handle to the same object and thus share it. An example of this method is shown in Figure 21.9.

```
// Process A
. . .
HANDLE hSemaphore = CreateSemaphore(NULL, 1, 1, L"MySEM1");
. . .

// Process B
. . .
HANDLE hSemaphore = OpenSemaphore(SEMAPHORE_ALL_ACCESS,
   FALSE, L"MySEM1");
. . .
```

Figure 21.8 Code for sharing an object by name lookup.

21.7.3 Process Management

In Windows, a **process** is a loaded instance of an application and a **thread** is an executable unit of code that can be scheduled by the kernel dispatcher. Thus, a process contains one or more threads. A process is created when a thread in some other process calls the `CreateProcess()` API. This routine loads any dynamic link libraries used by the process and creates an initial thread in the process. Additional threads can be created by the `CreateThread()` function.

```
// Process A wants to give Process B access to a semaphore

// Process A

DWORD dwProcessBId; // must; from some IPC mechanism
HANDLE hSemaphore = CreateSemaphore(NULL, 1, 1, NULL);
HANDLE hProcess = OpenProcess(PROCESS_DUP_HANDLE, FALSE,
   dwProcessBId);
HANDLE hSemaphoreCopy;
DuplicateHandle(GetCurrentProcess(), hSemaphore,
   hProcess, &hSemaphoreCopy,
   0, FALSE, DUPLICATE_SAME_ACCESS);
// send the value of the semaphore to Process B
// using a message or shared memory object
. . .

// Process B
HANDLE hSemaphore = // value of semaphore from message
// use hSemaphore to access the semaphore
. . .
```

Figure 21.9 Code for sharing an object by passing a handle.

Each thread is created with its own stack, which defaults to 1 MB unless otherwise specified in an argument to `CreateThread()`.

21.7.3.1 Scheduling Rule

Priorities in the Win32 environment are based on the native kernel (NT) scheduling model, but not all priority values may be chosen. The Win32 API uses six priority classes:

1. `IDLE_PRIORITY_CLASS` (NT priority level 4)
2. `BELOW_NORMAL_PRIORITY_CLASS` (NT priority level 6)
3. `NORMAL_PRIORITY_CLASS` (NT priority level 8)
4. `ABOVE_NORMAL_PRIORITY_CLASS` (NT priority level 10)
5. `HIGH_PRIORITY_CLASS` (NT priority level 13)
6. `REALTIME_PRIORITY_CLASS` (NT priority level 24)

Processes are typically members of the `NORMAL_PRIORITY_CLASS` unless the parent of the process was of the `IDLE_PRIORITY_CLASS` or another class was specified when `CreateProcess` was called. The priority class of a process is the default for all threads that execute in the process. It can be changed with the `SetPriorityClass()` function or by passing an argument to the `start` command. Only users with the ***increase scheduling priority*** privilege can move a process into the `REALTIME_PRIORITY_CLASS`. Administrators and power users have this privilege by default.

When a user is switching between interactive processes and workloads, the system needs to schedule the appropriate threads so as to provide good responsiveness, which leads to a shorter quantums of execution. Yet, once the user has chosen a particular process, a good amount of throughput from this particular process is also expected. For this reason, Windows has a special scheduling rule for processes not in the `REALTIME_PRIORITY_CLASS`. Windows distinguishes between the process associated with the foreground window on the screen and the other (background) processes. When a process moves into the foreground, Windows increases the scheduling quantum for all its threads by a factor of 3; CPU-bound threads in the foreground process will run three times longer than similar threads in background processes. Because server systems always operate with a much larger quantum than client systems—a factor of 6—this behavior is not enabled for server systems. For both types of systems, however, the scheduling parameters can be customized through the appropriate system dialog or registry key.

21.7.3.2 Thread Priorities

A thread starts with an initial priority determined by its class. The priority can be altered by the `SetThreadPriority()` function. This function takes an argument that specifies a priority relative to the base priority of its class:

- `THREAD_PRIORITY_LOWEST`: base − 2
- `THREAD_PRIORITY_BELOW_NORMAL`: base − 1

- THREAD_PRIORITY_NORMAL: base + 0
- THREAD_PRIORITY_ABOVE_NORMAL: base + 1
- THREAD_PRIORITY_HIGHEST: base + 2

Two other designations are also used to adjust the priority. Recall from Section 21.3.4.3 that the kernel has two priority classes: 16–31 for the static class and 1–15 for the variable class. THREAD_PRIORITY_IDLE sets the priority to 16 for static-priority threads and to 1 for variable-priority threads. THREAD_PRIORITY_TIME_CRITICAL sets the priority to 31 for real-time threads and to 15 for variable-priority threads.

The kernel adjusts the priority of a variable class thread dynamically depending on whether the thread is I/O bound or CPU bound. The Win32 API provides a method to disable this adjustment via SetProcessPriorityBoost() and SetThreadPriorityBoost() functions.

21.7.3.3 Thread Suspend and Resume

A thread can be created in a ***suspended state*** or can be placed in a suspended state later by use of the SuspendThread() function. Before a suspended thread can be scheduled by the kernel dispatcher, it must be moved out of the suspended state by use of the ResumeThread() function. Both functions set a counter so that if a thread is suspended twice, it must be resumed twice before it can run.

21.7.3.4 Thread Synchronization

To synchronize concurrent access to shared objects by threads, the kernel provides synchronization objects, such as semaphores and mutexes. These are dispatcher objects, as discussed in Section 21.3.4.3. Threads can also synchronize with kernel services operating on kernel objects—such as threads, processes, and files—because these are also dispatcher objects. Synchronization with kernel dispatcher objects can be achieved by use of the WaitForSingleObject() and WaitForMultipleObjects() functions; these functions wait for one or more dispatcher objects to be signaled.

Another method of synchronization is available to threads within the same process that want to execute code exclusively. The Win32 **critical section object** is a user-mode mutex object that can often be acquired and released without entering the kernel. On a multiprocessor, a Win32 critical section will attempt to spin while waiting for a critical section held by another thread to be released. If the spinning takes too long, the acquiring thread will allocate a kernel mutex and yield its CPU. Critical sections are particularly efficient because the kernel mutex is allocated only when there is contention and then used only after attempting to spin. Most mutexes in programs are never actually contended, so the savings are significant.

Before using a critical section, some thread in the process must call InitializeCriticalSection(). Each thread that wants to acquire the mutex calls EnterCriticalSection() and then later calls LeaveCriticalSection() to release the mutex. There is also a TryEnterCriticalSection() function, which attempts to acquire the mutex without blocking.

For programs that want user-mode reader–writer locks rather than mutexes, Win32 supports **slim reader–writer** (**SRW**) **locks**. SRW locks have APIs similar to those for critical sections, such as `InitializeSRWLock`, `AcquireSRWLockXXX`, and `ReleaseSRWLockXXX`, where XXX is either `Exclusive` or `Shared`, depending on whether the thread wants write access or only read access to the object protected by the lock. The Win32 API also supports **condition variables**, which can be used with either critical sections or SRW locks.

21.7.3.5 Thread Pool

Repeatedly creating and deleting threads can be expensive for applications and services that perform small amounts of work in each instantiation. The Win32 thread pool provides user-mode programs with three services: a queue to which work requests may be submitted (via the `SubmitThreadpoolWork()` function), an API that can be used to bind callbacks to waitable handles (`RegisterWaitForSingleObject()`), and APIs to work with timers (`CreateThreadpoolTimer()` and `WaitForThreadpoolTimerCallbacks()`) and to bind callbacks to I/O completion queues (`BindIoCompletionCallback()`).

The goal of using a thread pool is to increase performance and reduce memory footprint. Threads are relatively expensive, and each processor can be executing only one thread at a time no matter how many threads are available. The thread pool attempts to reduce the number of runnable threads by slightly delaying work requests (reusing each thread for many requests) while providing enough threads to effectively utilize the machine's CPUs. The wait and I/O- and timer-callback APIs allow the thread pool to further reduce the number of threads in a process, using far fewer threads than would be necessary if a process were to devote separate threads to servicing each waitable handle, timer, or completion port.

21.7.3.6 Fibers

A **fibe** is user-mode code that is scheduled according to a user-defined scheduling algorithm. Fibers are completely a user-mode facility; the kernel is not aware that they exist. The fiber mechanism uses Windows threads as if they were CPUs to execute the fibers. Fibers are cooperatively scheduled, meaning that they are never preempted but must explicitly yield the thread on which they are running. When a fiber yields a thread, another fiber can be scheduled on it by the run-time system (the programming language run-time code).

The system creates a fiber by calling either `ConvertThreadToFiber()` or `CreateFiber()`. The primary difference between these functions is that `CreateFiber()` does not begin executing the fiber that was created. To begin execution, the application must call `SwitchToFiber()`. The application can terminate a fiber by calling `DeleteFiber()`.

Fibers are not recommended for threads that use Win32 APIs rather than standard C-library functions because of potential incompatibilities. Win32 user-mode threads have a **thread-environment block** (**TEB**) that contains numerous per-thread fields used by the Win32 APIs. Fibers must share the TEB of the thread on which they are running. This can lead to problems when a Win32 interface puts state information into the TEB for one fiber and then the information is

overwritten by a different fiber. Fibers are included in the Win32 API to facilitate the porting of legacy UNIX applications that were written for a user-mode thread model such as Pthreads.

21.7.3.7 User-Mode Scheduling UMS and ConcRT

A new mechanism in Windows 7, user-mode scheduling (UMS), addressed several limitations of fibers. As just noted, fibers are unreliable for executing Win32 APIs because they do not have their own TEBs. When a thread running a fiber blocks in the kernel, the user scheduler loses control of the CPU for a time as the kernel dispatcher takes over scheduling. Problems may result when fibers change the kernel state of a thread, such as the priority or impersonation token, or when they start asynchronous I/O.

UMS provides an alternative model by recognizing that each Windows thread is actually two threads: a kernel thread (KT) and a user thread (UT). Each type of thread has its own stack and its own set of saved registers. The KT and UT appear as a single thread to the programmer because UTs can never block but must always enter the kernel, where an implicit switch to the corresponding KT takes place. UMS uses each UT's TEB to uniquely identify the UT. When a UT enters the kernel, an explicit switch is made to the KT that corresponds to the UT identified by the current TEB. The reason the kernel does not know which UT is running is that UTs can invoke a user-mode scheduler, as fibers do. But in UMS, the scheduler switches UTs, including switching the TEBs.

When a UT enters the kernel, its KT may block. When this happens, the kernel switches to a scheduling thread, which UMS calls a ***primary thread***, and uses this thread to reenter the user-mode scheduler so that it can pick another UT to run. Eventually, a blocked KT will complete its operation and be ready to return to user mode. Since UMS has already reentered the user-mode scheduler to run a different UT, UMS queues the UT corresponding to the completed KT to a completion list in user mode. When the user-mode scheduler is choosing a new UT to switch to, it can examine the completion list and treat any UT on the list as a candidate for scheduling. The key features of UMS are depicted in Figure 21.10.

Unlike fibers, UMS is not intended to be used directly by programmers. The details of writing user-mode schedulers can be very challenging, and UMS does not include such a scheduler. Rather, the schedulers come from programming language libraries that build on top of UMS. Microsoft Visual Studio 2010 shipped with Concurrency Runtime (ConcRT), a concurrent programming framework for C++. ConcRT provides a user-mode scheduler together with facilities for decomposing programs into tasks, which can then be scheduled on the available CPUs. ConcRT provides support for `par_for` styles of constructs, as well as rudimentary resource management and task synchronization primitives. However, as of Visual Studio 2013, the UMS scheduling mode is no longer available in ConcRT. Significant performance metrics showed that true parallel programs that are well written do not spend a large amount of time context-switching between their tasks. The benefits that UMS provided in this space did not outweigh the complexity of maintaining a separate scheduler—in some cases, even the default NT scheduler performed better.

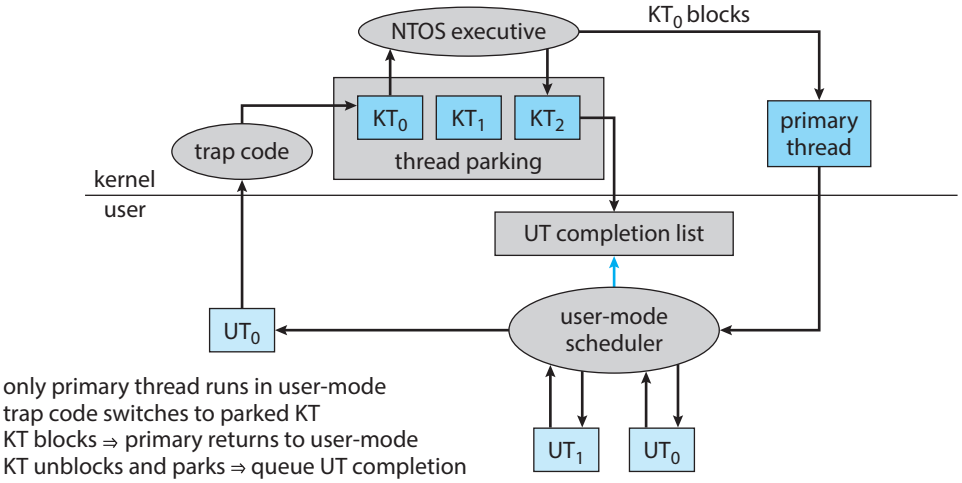


Figure 21.10 User-mode scheduling.

21.7.3.8 Winsock

Winsock is the Windows sockets API. Winsock is a session-layer interface that is largely compatible with BSD sockets but has some added Windows extensions. It provides a standardized interface to many transport protocols that may have different addressing schemes, so that any Winsock application can run on any Winsock-compliant protocol stack. Winsock underwent a major update in Windows Vista to add tracing, IPv6 support, impersonation, new security APIs, and many other features.

Winsock follows the Windows Open System Architecture (WOSA) model, which provides a standard service provider interface (SPI) between applications and networking protocols. Applications can load and unload ***layered protocols*** that build additional functionality, such as additional security, on top of the transport protocol layers. Winsock supports asynchronous operations and notifications, reliable multicasting, secure sockets, and kernel mode sockets. It also supports simpler usage models, like the `WSAConnectByName()` function, which accepts the target as strings specifying the name or IP address of the server and the service or port number of the destination port.

21.7.4 IPC Using Windows Messaging

Win32 applications handle interprocess communication in several ways. The typical high-performance way is by using local RPCs or named pipes. Another is by using shared kernel objects, such as named section objects, and a synchronization object, such as an event. Yet another is by using the Windows messaging facility—an approach that is particularly popular for Win32 GUI applications. One thread can send a message to another thread or to a window by calling `PostMessage()`, `PostThreadMessage()`, `SendMessage()`, `SendThreadMessage()`, or `SendMessageCallback()`. ***Posting*** a message and ***sending*** a message differ in this way: The post routines are asynchronous; they

return immediately, and the calling thread does not know when the message is actually delivered. The send routines are synchronous; they block the caller until the message has been delivered and processed.

In addition to sending a message, a thread can send data with the message. Since processes have separate address spaces, the data must be copied. The system copies data by calling `SendMessage()` to send a message of type `WM_COPYDATA` with a `COPYDATASTRUCT` data structure that contains the length and address of the data to be transferred. When the message is sent, Windows copies the data to a new block of memory and gives the virtual address of the new block to the receiving process.

Every Win32 GUI thread has its own input queue from which it receives messages. If a Win32 application does not call `GetMessage()` to handle events on its input queue, the queue fills up; and after about five seconds, the task manager marks the application as "Not Responding." Note that message passing is subject to the integrity level mechanism introduced earlier. Thus, a process may not send a message such as WM_COPYDATA to a process with a higher integrity level, unless a special Windows API is used to remove the protection (ChangeWindowMessageFilterEx).

21.7.5 Memory Management

The Win32 API provides several ways for an application to use memory: virtual memory, memory-mapped files, heaps, thread-local storage, and AWE physical memory.

21.7.5.1 Virtual Memory

An application calls `VirtualAlloc()` to reserve or commit virtual memory and `VirtualFree()` to de-commit or release the memory. These functions enable the application to specify the virtual address at which the memory is allocated. (Otherwise, a random address is selected, which is recommended for security reasons.) The functions operate on multiples of the memory page size but, for historical reasons, always return memory allocated on a 64-KB boundary. Examples of these functions appear in Figure 21.11. The `VirtualAllocEx()` and `VirtualFreeEx()` functions can be used to allocate and free memory in a separate process, while `VirtualAllocExNuma()` can be used to leverage memory locality on NUMA systems.

21.7.5.2 Memory-Mapped Files

Another way for an application to use memory is by memory-mapping a file into its address space. Memory mapping is also a convenient way for two processes to share memory: both processes map the same file into their virtual memory. Memory mapping is a multistage process, as you can see in the example in Figure 21.12.

If a process wants to map some address space just to share a memory region with another process, no file is needed. The process calls `CreateFileMapping()` with a file handle of `0xffffffff`, a particular size, and (optionally) a name. The resulting file-mapping object can be shared by inheritance, by name lookup (if it was named), or by handle duplication.

```
// reserve 16 MB at the top of our address space
PVOID pBuf = VirtualAlloc(NULL, 0x1000000,
    MEM_RESERVE | MEM_TOP_DOWN, PAGE_READWRITE);
// commit the upper 8 MB of the allocated space
VirtualAlloc((LPVOID)((DWORD_PTR)pBuf + 0x800000), 0x800000,
    MEM_COMMIT, PAGE_READWRITE);
// do something with the memory
. . .
// now decommit the memory
VirtualFree((LPVOID)((DWORD_PTR)pBuf + 0x800000), 0x800000,
    MEM_DECOMMIT);
// release all of the allocated address space
VirtualFree(pBuf, 0, MEM_RELEASE);
```

Figure 21.11 Code fragments for allocating virtual memory.

21.7.5.3 Heaps

Heaps provide a third way for applications to use memory, just as with `malloc()` and `free()` in standard C or `new()` and `delete()` in C++. A heap in the Win32 environment is a region of pre-committed address space. When a Win32 process is initialized, it is created with a **default heap**. Since most Win32

```
// set the file mapping size to 8MB
DWORD dwSize = 0x800000;
// open the file or create it if it does not exist
HANDLE hFile = CreateFile(L"somefile.ext",
   GENERIC_READ | GENERIC_WRITE,
   FILE_SHARE_READ | FILE_SHARE_WRITE, NULL,
   OPEN_ALWAYS, FILE_ATTRIBUTE_NORMAL, NULL);
// create the file mapping
HANDLE hMap = CreateFileMapping(hFile,
   PAGE_READWRITE | SEC_COMMIT, 0, dwSize, L"SHM_1");
// now get a view of the space mapped
PVOID pBuf = MapViewOfFile(hMap, FILE_MAP_ALL_ACCESS,
   0, 0, 0, dwSize);
// do something with the mapped file
. . .
// now unmap the file
UnmapViewOfFile(pBuf);
CloseHandle(hMap);
CloseHandle(hFile);
```

Figure 21.12 Code fragments for memory mapping of a file.

applications are multithreaded, access to the heap is synchronized to protect the heap's space-allocation data structures from being damaged by concurrent updates by multiple threads. The advantage of the heap is that it can be used to make allocations as small as 1 byte, because the underlying memory pages have already been committed. Unfortunately, heap memory cannot be shared or marked as read-only, because all heap allocations share the same pages. However, by using `HeapCreate()`, a programmer can create his or her own heap, which can be marked as read-only with `HeapProtect()`, created as an executable heap, or even allocated on a specific NUMA node.

Win32 provides several heap-management functions so that a process can allocate and manage a private heap. These functions are `HeapCreate()`, `HeapAlloc()`, `HeapRealloc()`, `HeapSize()`, `HeapFree()`, and `HeapDestroy()`. The Win32 API also provides the `HeapLock()` and `HeapUnlock()` functions to enable a thread to gain exclusive access to a heap. Note that these functions perform only synchronization; they do not truly "lock" pages against malicious or buggy code that bypasses the heap layer.

The original Win32 heap was optimized for efficient use of space. This led to significant problems with fragmentation of the address space for larger server programs that ran for long periods of time. A new **low-fragmentation heap** (**LFH**) design introduced in Windows XP greatly reduced the fragmentation problem. The heap manager in Windows 7 and later versions automatically turns on LFH as appropriate. Additionally, the heap is a primary target of attackers using vulnerabilities such as double-free, use-after-free, and other memory-corruption-related attacks. Each version of Windows, including Windows 10, has added more randomness, entropy, and security mitigations to prevent attackers from guessing the ordering, size, location, and content of heap allocations.

21.7.5.4 Thread-Local Storage

A fourth way for applications to use memory is through a **thread-local storage** (**TLS**) mechanism. Functions that rely on global or static data typically fail to work properly in a multithreaded environment. For instance, the C run-time function `strtok()` uses a static variable to keep track of its current position while parsing a string. For two concurrent threads to execute `strtok()` correctly, they need separate `current position` variables. TLS provides a way to maintain instances of variables that are global to the function being executed but not shared with any other thread.

TLS provides both dynamic and static methods of creating thread-local storage. The dynamic method is illustrated in Figure 21.13. The TLS mechanism allocates global heap storage and attaches it to the thread environment block (TEB) that Windows allocates to every user-mode thread. The TEB is readily accessible by each thread and is used not just for TLS but for all the per-thread state information in user mode.

21.7.5.5 AWE Memory

A final way for applications to use memory is through the **Address Windowing Extension** (**AWE**) functionality. This mechanism allows a developer to directly

```
// reserve a slot for a variable
DWORD dwVarIndex = T1sAlloc();
// make sure a slot was available
if (dwVarIndex == TLS_OUT_OF_INDEXES)
return;
// set it to the value 10
T1sSetValue(dwVarIndex, (LPVOID)10);
// get the value
DWORD dwVar = (DWORD)(DWORD_PTR)T1sGetValue(dwVarIndex);
// release the index
T1sFree(dwVarIndex);
```

Figure 21.13 Code for dynamic thread-local storage.

request free physical pages of RAM from the memory manager (through `AllocateUserPhysicalPages()`) and later commit virtual memory on top of the physical pages using `VirtualAlloc()`. By requesting various regions of physical memory (including scatter-gather support), a user-mode application can access more physical memory than virtual address space; this is useful on 32-bit systems, which may have more than 4 GB of RAM). In addition, the application can bypass the memory manager's caching, paging, and coloring algorithms. Similar to UMS, AWE may thus offer a way for certain applications to extract additional performance or customization beyond what Windows offers by default. SQL Server, for example, uses AWE memory.

To use a thread-local static variable, the application declares the variable as follows to ensure that every thread has its own private copy:

```
_declspec(thread) DWORD cur_pos = 0;
```

21.8 Summary

- Microsoft designed Windows to be an extensible, portable operating system—one able to take advantage of new techniques and hardware.
- Windows supports multiple operating environments and symmetric multiprocessing, including both 32-bit and 64-bit processors and NUMA computers.
- The use of kernel objects to provide basic services, along with support for client–server computing, enables Windows to support a wide variety of application environments.
- Windows provides virtual memory, integrated caching, and preemptive scheduling.

- To protect user data and guarantee program integrity, Windows supports elaborate security mechanisms and exploit mitigations and takes advantage of hardware virtualization.
- Windows runs on a wide variety of computers, so users can choose and upgrade hardware to match their budgets and performance requirements without needing to alter the applications they run.
- By including internationalization features, Windows can run in a variety of countries and many languages.
- Windows has sophisticated scheduling and memory-management algorithms for performance and scalability.
- Recent versions of Windows have added power management and fast sleep and wake features, and decreased resource use in several areas to be more useful on mobile systems such as phones and tablets.
- The Windows volume manager and NTFS file system provide a sophisticated set of features for desktop as well as server systems.
- The Win32 API programming environment is feature rich and expansive, allowing programmers to use all of Windows's features in their programs.

Practice Exercises

21.1 What type of operating system is Windows? Describe two of its major features.

21.2 List the design goals of Windows. Describe two in detail.

21.3 Describe the booting process for a Windows system.

21.4 Describe the three main architectural layers of the Windows kernel.

21.5 What is the job of the object manager?

21.6 What types of services does the process manager provide?

21.7 What is a local procedure call?

21.8 What are the responsibilities of the I/O manager?

21.9 What types of networking does Windows support? How does Windows implement transport protocols? Describe two networking protocols.

21.10 How is the NTFS namespace organized?

21.11 How does NTFS handle data structures? How does NTFS recover from a system crash? What is guaranteed after a recovery takes place?

21.12 How does Windows allocate user memory?

21.13 Describe some of the ways in which an application can use memory via the Win32 API.

Further Reading

[Russinovich et al. (2017)] give a deep overview of Windows 10 and considerable technical detail about system internals and components.

Bibliography

[Russinovich et al. (2017)] M. Russinovich, D. A. Solomon, and A. Ionescu, *Windows Internals–Part 1*, Seventh Edition, Microsoft Press (2017).

Chapter 21 Exercises

21.14 Under what circumstances would one use the deferred procedure calls facility in Windows?

21.15 What is a handle, and how does a process obtain a handle?

21.16 Describe the management scheme of the virtual memory manager. How does the VM manager improve performance?

21.17 Describe a useful application of the no-access page facility provided in Windows.

21.18 Describe the three techniques used for communicating data in a local procedure call. What settings are most conducive to the application of the different message-passing techniques?

21.19 What manages caching in Windows? How is the cache managed?

21.20 How does the NTFS directory structure differ from the directory structure used in UNIX operating systems?

21.21 What is a process, and how is it managed in Windows?

21.22 What is the fiber abstraction provided by Windows? How does it differ from the thread abstraction?

21.23 How does user-mode scheduling (UMS) in Windows 7 differ from fibers? What are some trade-offs between fibers and UMS?

21.24 UMS considers a thread to have two parts, a UT and a KT. How might it be useful to allow UTs to continue executing in parallel with their KTs?

21.25 What is the performance trade-off of allowing KTs and UTs to execute on different processors?

21.26 Why does the self-map occupy large amounts of virtual address space but no additional virtual memory?

21.27 How does the self-map make it easy for the VM manager to move the page-table pages to and from disk? Where are the page-table pages kept on disk?

21.28 When a Windows system hibernates, the system is powered off. Suppose you changed the CPU or the amount of RAM on a hibernating system. Do you think that would work? Why or why not?

21.29 Give an example showing how the use of a suspend count is helpful in suspending and resuming threads in Windows.